Data Formats: XML & JSON

Week 4 · Foundations module · Textbook Chapter 4

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 9 & 11, 2026

This week at a glance

Two formats carry almost all web data: **XML** (a tree of tags) and **JSON** (nested lists and dictionaries). This week we learn to parse both.

Monday | Labor Day — no class

- Monday's concept intro is **folded into Wednesday**

Wednesday | Concepts & notebook lab

- XML & JSON concepts, then the Chapter 4 notebook: Congress, tweets, and weather

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 4

Companion notebook

`ch-04-data-formats.ipynb`

Short week

No class Monday (Labor Day). We meet **Wednesday** (concepts + notebook) and **Friday** (revisions).

1. Monday · Concepts

Why data formats matter — No class Monday (Labor Day) — these concepts open Wednesday's session, just before the notebook.

Before you can *extract* data from the web, you have to understand how it is *structured*. Two formats dominate:

- **XML** — the older, document-centric format; a tree of nested tags. Born from the 1990s web.
- **JSON** — the newer, lighter format of the API economy; maps directly onto Python lists and dictionaries.

Every page, API response, and data feed you meet this semester is one of these — or HTML, which is XML's sibling.

When to use which

XML: older web services, government feeds (Congress, SEC, RSS), configuration files.

JSON: modern web APIs, database exports, service-to-service exchange.

Textbook Ch. 4, "Why Data Formats Matter" & "When to Use Which."

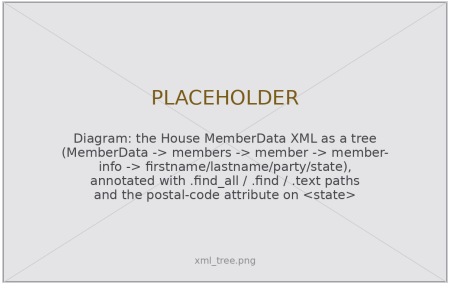
XML: a tree of nested tags

XML organizes data as a **hierarchy**: each tag can hold text, attributes, and child tags.

XML and HTML are **siblings**, not parent and child — both descend from SGML. Only the stricter XHTML dialect is well-formed XML; the HTML in the wild is not, which is exactly why the forgiving parsers of Ch. 6 exist.

A value can hide in one of two places:

- **text** — between an opening and closing tag (read with `.text`)
- **attributes** — key/value pairs inside the opening tag (read with brackets)



The House `MemberData` feed, drawn as a tree.

Textbook Ch. 4, "The Tree Structure." The siblings-with-HTML idea returns in Ch. 6.

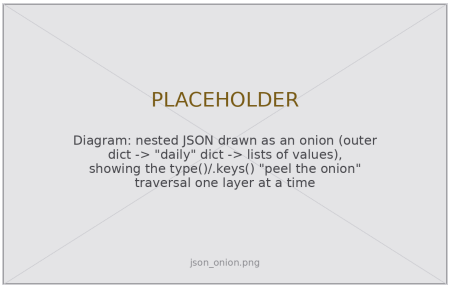
JSON: lists and dictionaries you already know

JSON maps onto two Python structures you already use:

- a JSON **array** is a Python **list** — ordered, indexed by position
- a JSON **object** is a Python **dict** — accessed by key

Real data **nests** them — dicts inside lists inside dicts, several levels deep. The skill is navigating from the outside in, one key or index at a time.

Created by Douglas Crockford in the early 2000s; the rise of AJAX made JSON the default API format by the 2010s.



"Peel the onion": `type()`, then `.keys()`, one layer deeper each time.

Textbook Ch. 4, "JSON and Python Data Structures."

One pipeline unifies the whole course

Whether data starts as XML, JSON, HTML, or PDF text, the move into analysis is always the same:

find records → **extract fields** →
list of dictionaries → `pd.DataFrame`

The DataFrame is the **universal intermediate format**. Reach it and all of pandas opens up: filter, group, cross-tabulate, merge.

The source changes; the tag names change; **the pipeline stays the same**.

You reuse this pattern in…

- HTML tables — Ch. 6
- archived pages — Ch. 7
- PDF text — Ch. 9
- Wikipedia, government & social APIs — Ch. 10–12

Textbook Ch. 4, "From XML to DataFrame." The same pattern recurs in every later chapter.

2. Wednesday · Notebook Lab

Wednesday — parsing XML with BeautifulSoup

Parse the string into a navigable tree, then search it. `.find()` returns the first match, `.find_all()` returns them all — each searches the *entire* subtree beneath a tag:

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(house_raw, "xml") # the "xml" parser comes from lxml

members = soup.find_all("member")
print(len(members)) # 2

for m in members:
    first = m.find("firstname").text # .text = content between the tags
    last = m.find("lastname").text # .find() reaches nested tags too
    party = m.find("party").text
    print(f"{first} {last} ({party})") # Joe Neguse (D) / Jeff Hurd (R)
```

Textbook Ch. 4, "Parsing with BeautifulSoup." Hyphenated names like `member-info` break dot-notation — use `.find()`.

The two-place problem — and the crash that guards it

A value is either **text** or an **attribute**:

```
state = soup.find("state")
state["postal-code"] # "CO"
state.find("state-fullname").text # "Colorado"
```

A missing tag makes `.find()` return `None`:

```
# AttributeError: None has no .text
m.find("party").text
# guard every extraction instead:
m.find("party").text if m.find("party") else None
```

Working out *which* of the two holds your value is half the parsing battle — real XML uses both.

Calling `.text` on a `None` is the **single most common** scraping error. The `if ...else None` guard keeps one missing field from crashing a 441-record loop.

XML always hands you **text**: converting `"2nd"` or `"$1,234"` to a number is your job.

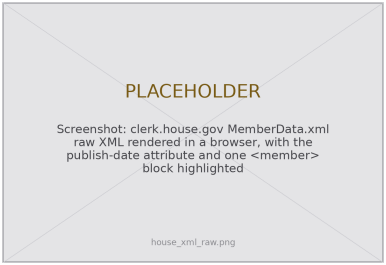
Textbook Ch. 4, "Parsing with BeautifulSoup" & "Common Issues to Debug."

XML → DataFrame: the real House roster

```
import requests, pandas as pd

url = "https://clerk.house.gov/xml/lists/MemberData.xml"
soup = BeautifulSoup(requests.get(url).text, "xml")
print(soup.find("MemberData")["publish-date"])

records = []
for m in soup.find_all("member"): # 441 members
    st = m.find("state")
    records.append({"last_name": m.find("lastname").text,
                    "party": m.find("party").text,
                    "state": st["postal-code"] if st else None})
df = pd.DataFrame(records) # find_all -> dicts -> DataFrame
```



The Clerk's live `MemberData.xml`.

Textbook Ch. 4, "From XML to DataFrame." 441 = 435 voting members + 6 delegates; record the `publish-date`.

JSON: parse a string, then the list-of-dictionaries pipeline

`json.loads()` turns a **s**tring into Python objects (`json.load()` reads a **f**ile). With `requests`, `response.json()` does it for you:

```
import json, pandas as pd

data = json.loads('{"state": "Colorado", "population": 5773714}')
print(data["state"]) # Colorado
# response.json() == json.loads(response.text)

census = [ # the list-of-dictionaries pattern:
    {"state": "Colorado", "year": 2020, "population": 5773714},
    {"state": "Colorado", "year": 2010, "population": 5029196},
]
df = pd.DataFrame(census) # one dict per row, keys -> columns
```

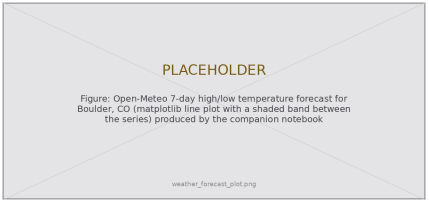
Textbook Ch. 4, "Parsing JSON Strings" & "The List-of-Dictionaries Pattern."

JSON: peel the onion on a live API

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {"latitude": 40.01, "longitude": -105.27,
          "daily": "temperature_2m_max",
          "timezone": "America/Denver"}
data = requests.get(url, params=params).json()

# type() + .keys(), one level deeper each time
data.keys() # latitude, longitude, daily...
data["daily"].keys() # time, temperature_2m_max
highs = data["daily"]["temperature_2m_max"] # a list
data.get("elevation", "unknown") # .get avoids KeyError
```



Boulder's 7-day forecast, straight from the nested lists.

Textbook Ch. 4, "Handling Deeply Nested JSON." `.get(key, default)` returns a default instead of raising `KeyError`.

Lab: work these in pairs, then show-and-tell

From the Chapter 4 notebook:

1. **RSS feed —** parse a news feed with the `"xml"` parser; pull title, link, pubDate → DataFrame
2. **Nested JSON —** Open-Meteo dates + max temps → DataFrame
3. **XML vs. JSON —** same data, both formats: which parses easier? which carries more metadata?
4. **Reusable —** write `xml_to_dataframe(soup, tag, fields)`; test on Congress, then an RSS feed
5. **Weather —** plot high & low temperatures; flag days that drop below freezing
6. 5617: a datasheet-style critique of a public XML/JSON dataset (Gebru et al. (2021))

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

The data source changes.
The tag names change.
The pipeline stays the same.

`find_all` → dicts → `DataFrame`.

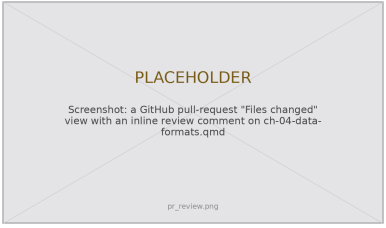
3. Friday · Textbook Revisions

Friday — improving Chapter 4 together

Today we make the book better. Each of you opens a **pull request** against `ch-04-data-formats.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 4

High-value targets — pick one and scope it to a single PR:

- **Test the code against live sources.** The House roster drifts (441 today; the `publish-date` changes) and Open-Meteo dates roll forward. Run the notebook, report what changed, and pin the snapshot.
- **Close a gap in "Common Issues to Debug."** Add a worked `json.JSONDecodeError` case — a trailing comma, single quotes, or an HTML error page returned instead of JSON.
- **Add a figure.** The chapter has *no* diagrams: contribute a clean XML-tree or a JSON "peel-the-onion" figure.
- **Strengthen an exercise.** Give the RSS or weather exercise expected output, a rubric, or a starter cell so it is self-checking.
- **Extend the RSS / post-API story.** Tie the decline-of-RSS narrative to a concrete citation and the "soft enclosure" theme (Ch. 3).

Targets drawn from the chapter's callouts, "Common Issues," exercises & "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Renders (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Web Architecture & Protocols** — how a URL becomes a rendered page (HTTP, requests, status codes).

Key takeaways

1. **XML** is a tree of tags; **JSON** is nested lists and dictionaries — the two structural languages of web data.
2. BeautifulSoup navigates XML with `.find()`, `.find_all()`, and `.text`; attributes come out with bracket notation.
3. For JSON, **peel the onion** — inspect `type()` and `.keys()` top-down, and use `.get()` for safe access.
4. The `find_all` → list-of-dicts → `DataFrame` pipeline is the bridge from raw web data to analysis.
5. Format is a **transparency** choice: open XML/JSON serves public oversight; scanned PDFs quietly enclose it.

Reading: Textbook Ch. 4. Related: Mitchell (2018); Vanden Broucke et al. (2018).